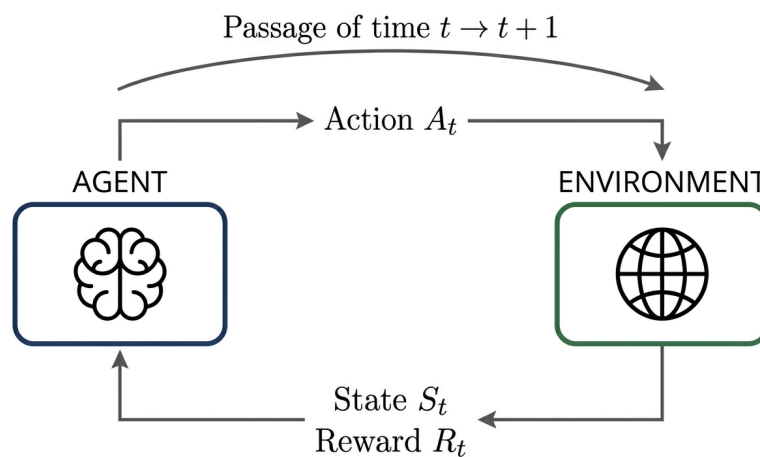


Reinforcement learning

General idea

Reinforcement learning is inspired by **research on psychology** and **animal training**

- An **agent interacts** with an **environment**
 - **Can influence** the **state** of the **environment**
 - **Obtains** numeric **reward signals**
- **Based on** the **state** returned by the environment, **the agent takes action**
 - **Policy: mapping** from **states to actions**
 - → The agent's **aim** is to **learn a policy** to **maximize** the **cumulative reward**



MDP - Markov Decision process

A **Markov Decision process** (MDP) is a framework that **helps** to **make decisions** on a **stochastic environment**

It is based on the following **components**:

- **States** s , beginning with the initial state s_0
- **Actions** a
- **Transition model** $P(s' | s, a)$
 - Markov assumption → the probability of going to s' from s only depends on actual state s
- **Reward function** $r(s)$
- **Policy** $\pi(s)$, defines the action an agent decides to do

A **MVP** can be **defined as** the following **tuple**:

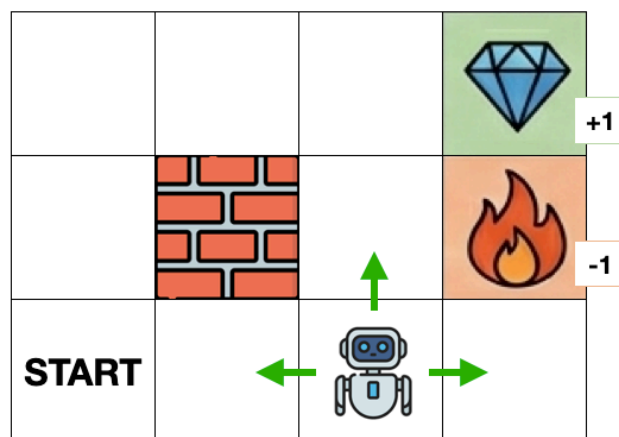
$$(S, A, R, P, \gamma)$$

Where:

- S is the set of **possible states**
- A is the set of **possible actions**
- R is the **distribution of the reward**, given a $(s \in S, a \in A)$ pair
- P is the **transition probability**, given a $(s \in S, a \in A)$ pair
- γ is the **discount factor** used to compute the cumulative reward

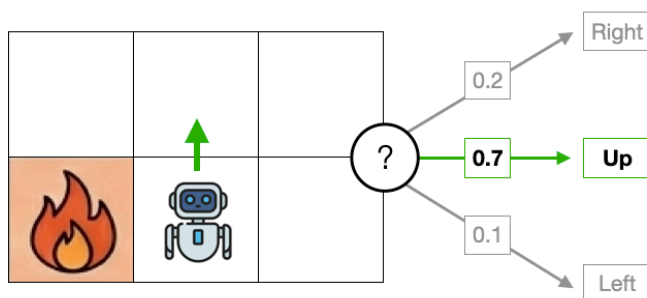
GRID WORLD EXAMPLE

Objective: reach the diamond terminal state in the least number of actions



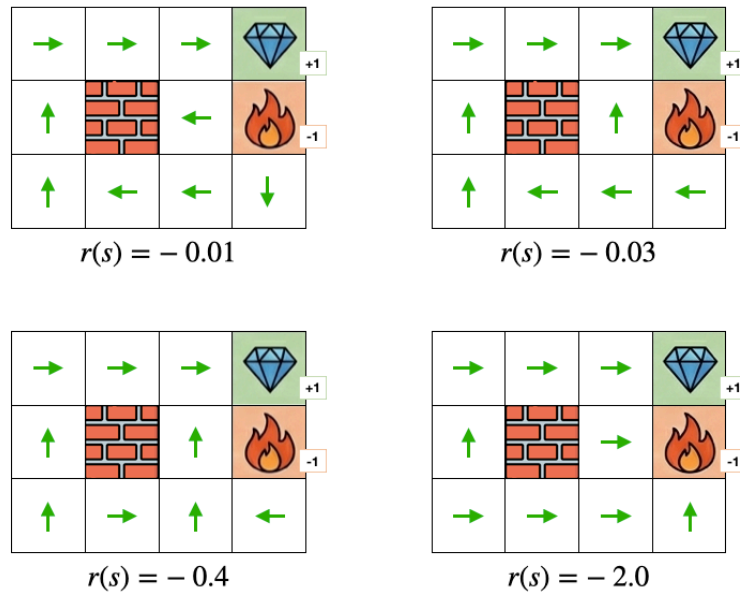
$$r(s) = -0.04 \text{ for every non-terminal state}$$

In a MVP environment **each step couple** $(s \in S, a \in A)$ **is associated** with a **probability**



In the example the probability to gain a higher reward moving from the actual state is higher if the taken action is moving up

Optimal policies depend on the reward function



MDP Loop

An **MDP follows** the following steps:

- At time $t = 0$ the **environment samples the starting state** s_0 from the probability $p(s_0)$
- **Repeat:**
 - The **agent observes** and **selects** an **action** a_t
 - The **environment:**
 - **Processes** the **action** $\rightarrow$ provides the **immediate reward** r_t based on $R(\cdot \mid s_t, a_t)$
 - **Computes** the **next state** $s_{t+1} \rightarrow$ based on **transition probability** $P(\cdot \mid s_t, a_t)$
 - The **agent receives** the **reward** r_t e the **new state** s_{t+1}

The **policy** $\pi(s)$ is a **function** from S to A that **specifies which action to take** in **each state**

- **Objective:** find **policy** π^* that **maximizes** the **cumulative discounted reward**

Cumulative discounted reward

Assume a policy π starting in state s_0 that leads to a sequence $s_0, s_1, s_2, \dots$

The **cumulative reward** of the **sequence** is the **sum of rewards** at each step:

$$\sum_{t \geq 0} r(s_t)$$

However, sequences **can vary** in **length** or even be **infinite**

- **Usually** the **cumulative reward** is a **sum discounted** by a factor γ

$$r(s_0) + \gamma r(s_1) + \gamma^2 r(s_2) + \dots + \gamma^T r(s_T) = \sum_{t \geq 0} \gamma^t r(s_t) \quad \text{with } 0 < \gamma \leq 1$$

- **Controls** the **importance** of future rewards
 - The **lower** the **discount factor** is, the **less important** future rewards are
 - The **agent** will **tend** to take **action** with **immediate rewards only**
- **Helps** the algorithm **to converge**

Reinforcement learning vs Supervised learning

Supervised learning

- The next **input sampled** **doesn't depend** on **previous inputs** or **agent predictions**
- There is a **supervision signal** at **every step**
- The **loss** is **differentiable**

Reinforcement learning

- Agent's **actions** **affect** the **environment** and **help** to **determine** the **next observation**
- **Rewards** may be **sparse**
- **Rewards** are **not differentiable**